CODEIT AI ENGINEER · 고급 프로젝트 (개인)

인디게임 개발사를 위한 광고 콘텐츠 생성 서비스

게임을 한 번 등록하면 스크린샷·요청 한 줄·굿즈 사진만으로 홍보 문구, 이벤트 배너, 게임 그림체 홍보 이미지, 굿즈 목업, 연출 사진, 상세 페이지까지 만드는 생성형 AI 서비스입니다. 모든 모델은 로컬 GPU 한 장(RTX 3090 24GB)에서 돕니다.









작성 **권순균** 기간 **2026-10-15 ~ 2026-11-13** 실험 노트북 **29개 (00~28)**

CONTENTS

목차

1. 한눈에 보기 — 문제 · 해결 · 핵심 결과
2. 서비스 기획 — 사용자 · 예시 게임 · 기능
3. 서비스 구조와 화면 — 구조도 · 작업 대기열 · 앱 화면
4. 데이터 준비와 탐색 — 03 · 04
5. 기능별 구현과 의사결정 — A · B · C · F · D · 상세 페이지
6. 서비스 처리 시간 — 21 단계별 묶음 처리
7. 사용성 자체 평가 — 가이드 6개 항목 · 28
8. 성과 · 한계 · 개선 방향
9. 진행 과정 — 날짜별 주요 결정

부록 A. 실험 노트북 00~28 목록

부록 B. 실행 환경과 실행 방법

부록 C. 출처 간 숫자 차이

읽는 법

- 숫자 뒤 괄호의 두 자리 번호는 실험 노트북 번호(`experiments/번호_이름.ipynb`)이고, 파일 이름은 `outputs/` 아래 기록 파일입니다.
- 점수 지표: **규칙 전부 통과**는 문구 규칙(형식·꼭 넣을 문구·판촉 표현·상투 구절 등)을 모두 지킨 비율, **그림체 점수(style_sim)**는 게임 원본 이미지와의 CLIP 평균 유사도, **SSIM**은 원본과의 구조 유사도(1이면 같음)입니다.
- 예시 결과 그림은 시연 때 새로 등록한 게임 "리치의 모험"과 예시 게임 3개(냥기사의 여행, KayKit, 0x72)의 실제 출력입니다.

01

한눈에 보기

문제

게임과 굿즈를 함께 파는 1인 인디게임 개발사는 디자인·마케팅 인력 없이 스토어 소개 문구, 이벤트 배너, 굿즈 판매 페이지를 직접 만들어야 합니다. 범용 생성 모델은 게임마다 다른 그림체(2D 애니풍·3D 로우폴리·픽셀)를 지키지 못하고, 굿즈 사진을 다시 그리면 실물과 달라집니다.

해결

게임을 한 번 등록(소개·판매 포인트·레퍼런스 아트·선택으로 LoRA)하면 6개 기능이 같은 게임 정보와 그림체 설정을 씁니다. 문구는 로컬 VLM + 코드 검증, 그림은 IP-Adapter(+LoRA), 굿즈는 원본 픽셀을 지키는 합성·조명 옮기기로 만들고, 요청은 작업 대기열이 모델별로 묶어 처리합니다.

핵심 결과

A 홍보 문구 · 규칙 전부 통과

0.40 → 1.00 KayKit

0x72 0.667 → 1.000 · 10 kk_must_summary.csv, 25 m25_by_game.csv

C 그림체 점수 · IP-Adapter 켬

0.552 → 0.646 KayKit

냥기사 LoRA 50장 + IP-Adapter 0.737 · 12 c_summary.csv, 18

F 굿즈 목업 · 인쇄 영역 밖 보존 SSIM

1.000 장당 0.3초

img2img 방식 0.909·약 6초 · 14 f_summary.csv

D 연출 사진 · 굿즈 보존 SSIM

0.842 → 0.993

IC-Light만 → 조명만 옮기기(R1n) · 16 r_summary.csv

상세 페이지 1장 처리 시간

270.5 → 80.9초

페이지별 모델 로드 → 단계별 묶음 처리 (3.35배) · 21 dp_service_time.csv

시연 (게임 등록 → 상세 페이지)

7 / 7 요청 완료

12분 53초 녹화, 요청당 처리 38.60~111.68초 (모델 단계 합) · 28 ue_stages.csv

한 번의 흐름으로 만든 결과 (시연 게임 "리치의 모험")



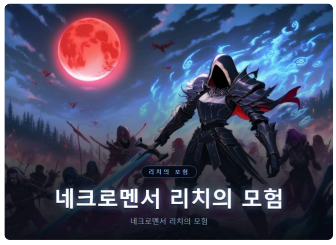
B 이벤트 배너 (요청 #23)



F 굿즈 목업 F1curve (#25)



D 연출 사진 YR1n·카페 (#26)



상세 페이지 860px (#45)

A 홍보 문구 (#21, 꼭 넣을 문구 "신규출시게임")

store: 붉은 달 아래 네크로맨서 리치가 무기를 휘두르는 야수적인 모험, 신규출시게임으로 여러분의 화면을 장악합니다. / 사람이 아닌 죽음의 군대를 이겨내는 긴장감 넘치는 전투를 경험해 보세요.

sns: 붉은 달 아래 네크로맨서 리치가 무기를 휘두르며 죽음의 군대를 이겨내는 신규출시게임, 지금 바로 만나보세요. (outputs/app/app.db 요청 #21)

02

서비스 기획

사용자

가이드가 고정한 사용자 "소상공인(광고·콘텐츠 제작에 인력·예산상 어려움을 겪는 사업자)" 안에서, **게임과 게임 굿즈를 함께 판매하는 1인 인디게임 개발사**로 좁혔습니다. 디지털 상품(게임)의 홍보 콘텐츠와 실물 상품(굿즈)의 판매 콘텐츠를 모두 필요로 하고, 두 콘텐츠 모두 게임의 그림체를 지켜야 한다는 공통 조건이 있습니다.

예시로 등록한 가상 게임 3개

실제 게임을 만들지 않고, 그림체가 서로 다른 오픈 라이선스 아트와 NovelAI로 직접 만든 아트를 가상의 게임으로 등록했습니다. 그림체가 다른 세 게임에서 같은 방식이 통하는지를 매 실험에서 함께 확인했습니다.

예시 게임	그림체	데이터
냥기사의 여행 (주력)	2D 애니풍 횡스크롤	NovelAI로 만든 아트 17장 + LoRA 학습용 800장
KayKit	3D 로우폴리 판타지 (CC0)	캐릭터 모델 Blender 렌더 520장
0x72 DungeonTileset II	16비트 픽셀 던전 (CC0)	타일·스프라이트를 조합한 장면 520장



냥기사의 여행 (07 최종 설정)



KayKit (07 최종 설정)



0x72 (07 최종 설정)

기능과 가이드 "구현 기능 예시" 대응

기능	입력 → 출력	가이드 예시
A 스크린샷 → 홍보 문구	스크린샷, 판매 포인트, 꼭 넣을 문구 → 스토어 소개 2문장 + SNS 1문장	상품 이미지에 어울리는 광고 문구 제안
B 텍스트 → 이벤트 배너	한국어 요청 한 문장 → 글자를 얹은 배너	광고 이미지 1: 텍스트 입력만
C 게임 그림체 홍보 이미지	장면 요청 + 등록한 레퍼런스 아트 → 같은 그림체의 새 장면	광고 이미지 2: 입력 이미지를 레퍼런스로 사용
F 굿즈 목업	무지 굿즈 사진 + 게임 그림 → 인쇄된 굿즈 사진	광고 이미지 3: 입력 이미지를 보존
D 굿즈 보존 연출 사진	굿즈 사진 + 장면 → 배경·조명만 바꾼 사진	광고 이미지 3: 입력 이미지를 보존
상세 페이지	앞 기능 결과를 구역별로 골라 가로 860px 한 장	서비스 산출물 "상세 페이지용 시각 자료"

범위 결정

- **로컬 모델로 구현 (10/22)**: 가이드에 안내된 OpenAI API가 실제로 제공되지 않아 Claude API로 기준선을 만들었고, API를 그대로 쓰면 직접 구현할 부분이 없으므로 로컬 VLM으로 비슷한 수준을 만드는 것을 목표로 했습니다. Claude 결과는 비교 기준선으로만 씁니다(02·08).
- **F를 깊게, D는 기본 방식만 (10/28)**: 개인 프로젝트 범위 안에서 굿즈 목업(F)에 시간을 집중했습니다.
- **사용성 평가 (11/04)**: 다른 사람에게 써 보게 하는 평가 대신, 시연 녹화와 앱 기록으로 가이드 6개 항목을 혼자 평가했습니다(28).

03

서비스 구조와 화면

GPU 한 장(24GB)에 모델을 모두 올려 둘 수 없습니다(Qwen3-VL-8B 최대 18.23GB + animagine 최대 13.0GB, 21). 그래서 화면은 요청만 받아 작업 대기열에 넣고, 워커가 요청을 모델 단계별로 묶어 단계마다 별도 프로세스에서 모델을 한 번만 올려 처리합니다.

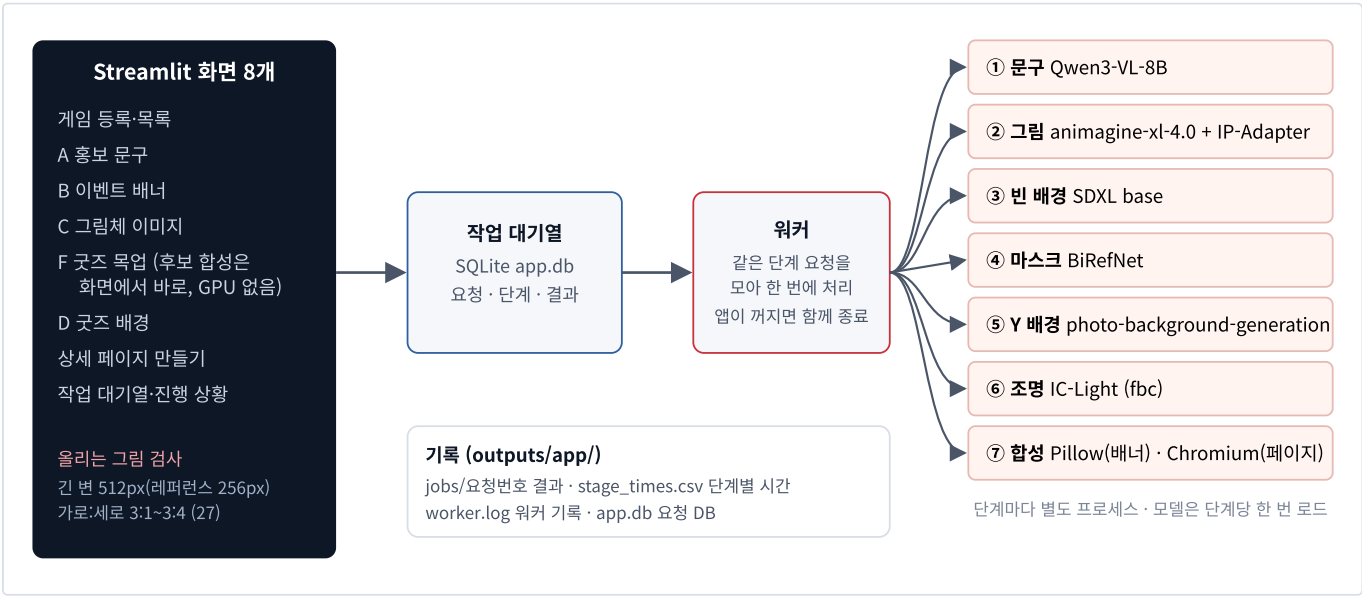
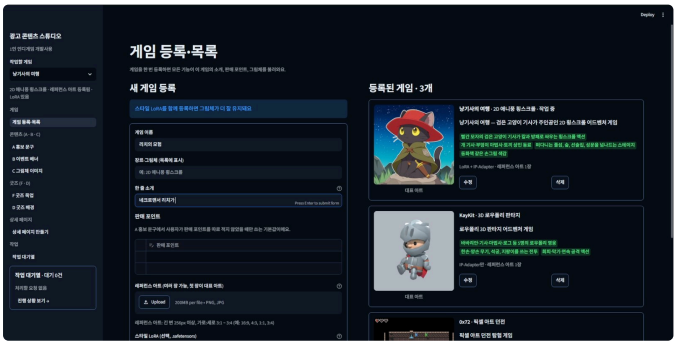
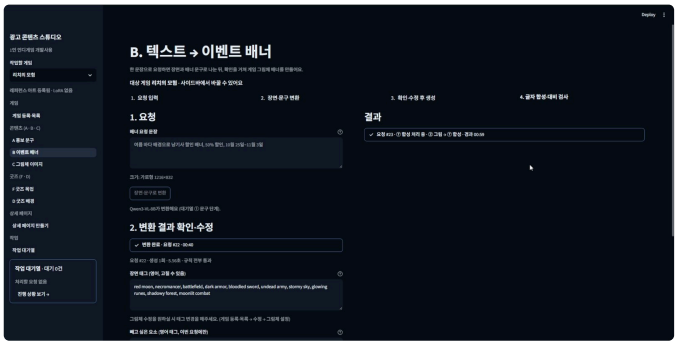


그림 3-1. 서비스 구조. 지시문·검증 규칙·모델 설정은 실험 노트북에서 채택한 코드를 그대로 옮겼습니다(README "서비스 앱").



게임 등록·목록: 예시 게임 3개가 처음부터 등록됨



B: 요청 → 영어 장면 태그·문구 변환 결과를 확인·수정한 뒤 생성



F: 굿즈 사진 + 디자인 → 후보를 모두 만들고 하나 고름



작업 대기열: 단계별 진행 상황과 요청 목록

화면: 11/04 시연 녹화(14:21~14:34)에서 캡처

04

데이터 준비와 탐색

데이터 구성

게임	원본	학습·비교용 데이터
냥기사의 여행	NAI 생성 이미지 17장, 1024×1024	NAI API로 생성 800장 (04)
0x72	애니메이션 프레임 210장, 16~32px	타일·스프라이트를 조합한 장면 520장
KayKit	미리보기 렌더 6장, 1920×1080	캐릭터 모델 Blender 렌더 520장
무지 굿즈 사진	-	무료 사진 사이트 11장 (Unsplash 9, Pexels 2): 티셔츠 2, 머그 4, 토트백 3, 후드 2

탐색으로 정한 것 (03)

발견	그래서 한 일
학습 데이터 1,840장 모두 1024×1024. NAI 800장의 알파 채널은 최솟값 254, 픽셀의 0.48~0.55%만 255 미만이고 `stealth_pngcomp`로 시작	투명 영역이 아니라 NAI가 넣은 생성 정보로 판단해 RGB로 바꿔 사용 (화면은 같음)
IP-Adapter 전처리는 짧은 변 224로 줄이고 가운데를 자름	정사각형 데이터라 크롭 손실이 없음을 확인
밝기·채도 평균±2표준편차로 거르면 NAI 800장 중 52장이 제외 후보인데, 그중 69.2%가 밤 장면이고, 그림체가 다른 KayKit의 26.7%는 범위 안	이 기준은 그림체가 아니라 장면 밝기를 거르므로 쓰지 않음
가장 가까운 이미지와의 CLIP 유사도 평균 KayKit 0.959, NAI 0.914, 0x72 0.981. KayKit은 같은 캐릭터·동작을 카메라 각도 4개로 렌더	나눠서 평가할 때 같은 묶음을 함께 둬야 함을 기록
묶음별 CLIP 임베딩 평균에 가장 가까운 이미지	IP-Adapter 참조 이미지로 사용: NAI `gen_0053`, KayKit `kk_0185`, 0x72 `x72_scene_0454` (05)

냥기사 LoRA 학습 데이터 생성 (04)

- 기존 17장의 PNG 메타데이터와 같은 생성 조건으로 800장을 만들었습니다. 캐릭터 고정 태그와 그림체 태그는 같게 두고 행동·장소·시간대·구도만 바꿨습니다.
- 구성: 기존 비율(주인공 6 : 조연·적 7 : 배경 4 : 스크린샷풍 2 : 아이템 1)을 늘려 주인공 240, 배경 160, 스크린샷 80, 동료·적 4종 각 70, 아이템 40장. 평가용 장면 문구는 학습 프롬프트에 넣지 않았습니다.
- 결과 800/800장. 이후 11에서 학습 장수를 50~800장으로 바꿔 비교한 결과, 50장으로도 800장과 같은 수준이었습니다(그림체 0.737 vs 0.736, 18).

출처: Notion "데이터 준비 (03 EDA · 04 LoRA 학습 데이터)", 03·04 노트북, goods_sources.csv

05 · A

기능 A · 스크린샷 → 홍보 문구

입력
게임 스크린샷, 판매 포인트, 꼭 넣을 문구, 말투

→

Qwen3-VL-8B
문구 규칙을 넣은 지시문

→

코드 검증
형식·꼭 넣을 문구·판촉 표현·상투 구절·sns/store 겹침

→

실패하면
샘플링 후보 4개 → 편집 지시 재요청 1번

문제

- Claude Haiku·Sonnet·Opus 기준선 문구 45개 중 "모험" 44, "떠나" 44, "~보세요" 43개 → 양산형 문구 (02).
- 꼭 넣을 문구가 있으면 형식이 무너짐: 형식 준수 Qwen 0.27. 문구를 문장 뒤에 따로 붙임("…헤쳐나가보세요. 스팀 찜하기") (08).

비교와 결정

단계	비교한 후보	결과	결정
모델 (08)	Qwen3-VL-8B, gemma-3-4b, Phi-4-multimodal, kanana-1.5-v-3b + Claude 기준선	Qwen 형식 준수 1.00·상투 구절 0.07·꼭 넣을 문구 유지 1.00, B 변환 지표 모두 1.00 (kanana 형식 0.00, gemma 문구 유지 0.07)	Qwen3-VL-8B 하나로 A·B 운영
꼭 넣을 문구 (08)	지시문 보강, 검증 후 재요청, 문구 칸 분리	칸 분리는 구조상 문장 밖에 붙음	문장 안에 넣고 코드로 검증
재생성 (08·09)	(가) 실패 이유 재요청 / (나) 샘플링 후보 여러 개	(가) 0.33, 같은 글 반복 8~11번 / (나) 후보 2개 0.42, 후보 4개 0.58	후보 4개
KayKit 실패 (10·13)	① 지시문 예시 ② 구체적 재요청 ③ ①+②	③ KayKit 0.40 → 1.00, 냥기사 0.867, 0x72 0.800	③ 세 게임 공통
후보 수 (23)	후보 4개 vs 8개	세 게임 평균 0.889 → 0.844 (0x72 0.8 → 0.533)	4개 유지
따로 붙는 실패 (25)	**미리 채우기, 토큰 막기, 편집 지시, sns만 다시 쓰기, 빈칸 틀, 제약 빔서치**	**편집 지시 0.956 (0x72 1.000, 재요청 단계 9/9). 빈칸 틀 1.000이지만 KayKit 15건 중 9건이 같은 뒷부분**	**편집 지시를 재요청에 적용**

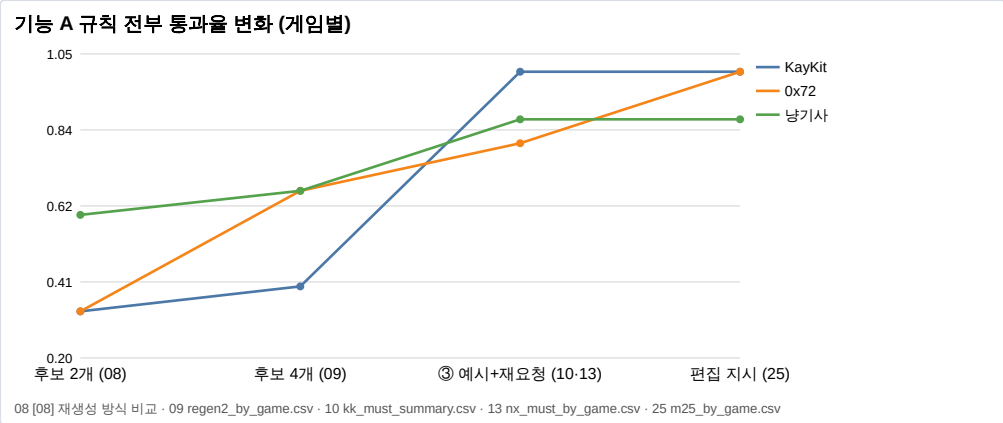


그림 5-1. 단계별 규칙 전부 통과율 (게임마다 5장 × seed 3개). KayKit은 ③에서, 0x72는 25 편집 지시에서 1.00이 됐습니다.

얻은 것 / 포기한 것 상투 구절 Claude 0.87~1.00 → Qwen 0.07, API 비용 대신 로컬 실행. 대신 건당 시간은 Claude 3.15~5.73초 → Qwen 22.5~26.0초이고, Qwen 최대 VRAM 16.8~18.22GB라 이미지 모델과 차례로 올려야 합니다(02·08·10·13). "꼭 넣을 문구는 문장 안에 자연스럽게" 규칙을 지키려고 문장 끝 붙이기 허용(0.73)을 택하지 않았습니다(09).

05 · B

기능 B · 텍스트 → 이벤트 배너

입력
한국어 요청 한 문장

→

Qwen3-VL-8B
영어 장면 태그 + 제목 + 부제
(확인·수정 가능)

→

animagine-xl-4.0
+ IP-Adapter, 게임별 프롬프트

→

Pillow 글자 합성
WCAG 2.1 대비 검사

결정과 근거

결정	근거
글자는 생성 모델이 아니라 Pillow로 합성	SDXL은 긴 글자를 읽을 수 있게 그리기 어렵고 무작위 글자가 섞임(SDXL 논문 Limitations, 07 첫 결과의 0x72 배너). Pillow 합성은 12장 모두 WCAG 대비 통과, 장당 0.03~0.04초 (07)
게임별 프롬프트로 문제 4가지 수정	23조건 × 92장 비교 (07): 냥기사 77토큰 초과 → 태그 줄이기(두 마리 0/4, 장면 0.148 → 0.193), KayKit 애니풍 4/4 → low poly 태그·부정 태그 0/4, 0x72 용암 → Compel 가중치 1.3에서만 4장 모두 표시
rating 태그 safe / 부정 nsfw	0x72 용암 요청 노출 캐릭터 5/8 → 0/8, 그림체·장면 점수 차이는 표준편차 안 (07 F2)
문구 규칙 위반 시 실패 이유를 알려 재요청, 동의어 허용	첫 문구 실패 3/36 → 0/36, 끝까지 남은 위반 0건. 금지어 삭제 방식은 문구가 끊김 (07 R3)



KayKit 수정 전: 전체 층 IP-Adapter → 애니풍 소녀 기사 (07 K)



수정 후: low poly 태그 + 부정 anime·1girl (07 K (a))



앱 결과: 냥기사 배너 "2027년 2월 출시" (LoRA 사용, 요청 #41)



앱 결과: 시연 게임 리치의 모험 배너 (요청 #23)

얻은 것 / 포기한 것 무작위 글자 없는 배너와 판촉 표현 위반 0건을 얻었고, 글자를 그림과 어우러지게 그리는 연출은 포기했습니다. KayKit은 애니풍을 없애는 대신 CLIP 그림체 점수가 0.704 → 0.626으로 내려갔는데, 애니풍인 쪽이 점수가 더 높아 이 경우는 눈으로 확인해 정했습니다(07).

05 · C

기능 C · 게임 그림체를 지키는 홍보 이미지

베이스 모델 (01 → 05)

이미지 생성 모델 13개를 텍스트만으로 만들어 보니 모두 게임 그림체에서 벗어났고(01), IP-Adapter가 붙는 6개를 게임 3개 × 강도 3 × seed 3으로 비교했습니다(05, 162장).

모델	냥기사 그림체	KayKit	0x72	장당 시간	VRAM
sdxl_base	0.738~0.739	0.786~0.814	0.878~0.887	16.1~17.1초	13.07GB
animagine_xl4 (채택)	**0.715~0.728**	**0.770~0.791**	**0.797~0.845**	**16.8~17.2초**	**13.07GB**
playground_v25	0.718~0.737	0.680~0.714	0.720~0.761	16.3~16.8초	13.07GB
sd35_large	0.740~0.754	0.804~0.817	0.864~0.893	202.5~220.3초	오프로드

animagine_xl4는 냥기사 그림체가 게임 내부 기준선(0.711) 이상이면서 오프로드 없이 장당 약 17초입니다. 세 게임을 한 모델로 운영한 이유는 IP-Adapter 파이프라인이 13.07GB라 24GB에 두 모델을 함께 올릴 수 없기 때문입니다. KayKit·0x72는 sdxl_base 점수가 더 높은 것을 포기한 값으로 기록했습니다(05).

IP-Adapter와 냥기사 LoRA (06 · 11 · 12 · 18)

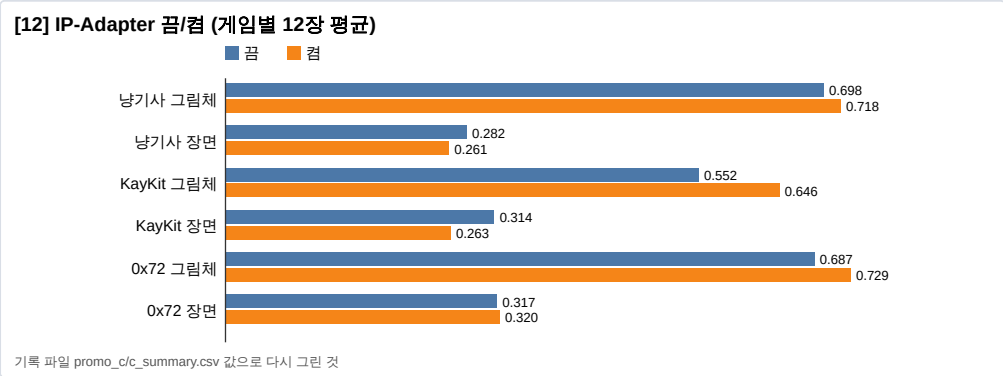


그림 5-2. IP-Adapter 끔/켬 (12, 장면 18건 × seed 2)

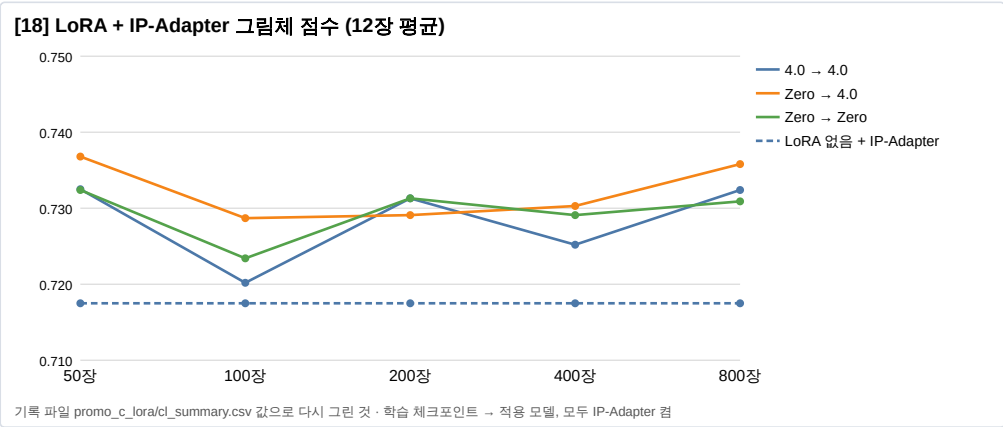


그림 5-3. 냥기사 LoRA 학습 장수별 + IP-Adapter (18)

- IP-Adapter를 켜면 세 게임 모두 그림체 점수가 오르고(+0.020~+0.094), KayKit·냥기사는 장면 점수가 내려갑니다(12).
- LoRA는 Zero 체크포인트에서 학습해 4.0에 적용한 쪽이 8개 조건 모두 높았고(06), 학습 장수 50~800장의 차이는 표준편차(0.010~0.018) 안이었습니다(11). IP-Adapter와 함께 쓰면 그림체 0.737(IP-Adapter만 0.718), 50장 0.737 = 800장 0.736 → **50장 LoRA(zero_trigger_n50) + IP-Adapter**로 정했습니다(18). 1인 개발자가 직접 모을 수 있는 장수라는 점이 기준이었습니다.
- 새로 등록한 게임의 기본 설정(게임 태그 없음)은 그림체 점수가 0.031~0.079 낮고, 냥기사 12장 중 7장이 사람 소녀, KayKit 12장 모두 애니풍 인물이었습니다(24) → 앱의 게임 등록 화면에 그림체 태그 칸과 스타일 LoRA·트리거 단어 등록 칸을 넣었습니다.

05 · F

기능 F · 굿즈 목업

무지 굿즈 사진의 모양·배경은 그대로 두고 게임 그림만 굿즈 위에 인쇄된 것처럼 얹는 것이 목표입니다(가이드 "광고 이미지 3: 입력 이미지를 보존").

방식 (14)	인쇄 영역 밖 보존 SSIM	굿즈 보존 SSIM	그림 유사도	장당 시간
F0 SDXL img2img + IP-Adapter, strength 0.45	0.909	0.806	0.530	약 6초
F1 합성 (원통 매핑 + 밝기 변위 + 곱하기 음영)	**1.000**	**1.000**	**0.941**	**0.3초**
F2 Qwen-Image-Edit-2509 (4bit, 오프로드)	0.279	0.159	0.482	596초
F3 F1 + SDXL 인페인팅 0.3	1.000	1.000	0.658	3.2초

결정 변경 10/28에는 "img2img가 입력 구도·형태를 유지한다"는 문서 근거로 F0를 골랐습니다. 21에서 상세 페이지를 이어 만들어 보니 F0 목업에는 그림이 거의 찍히지 않았고(그림 유사도 0.530과 같은 현상), 11/02 F1 합성 계열로 바꿨습니다. F1이 스티커처럼 보이는 문제는 22에서 위아래 휨(F1curve), 반사광(F1spec), 천 질감(F1cloth)을 더해 비교했고, 모든 후보가 장당 0.31~0.35초라 **후보를 모두 만들어 이용자가 고르게** 했습니다. 그림 유사도는 평평할수록 높아 사실감 비교에 쓸 수 없기 때문입니다(22).



무지 굿즈 사진 mug_01



F1curve · 냥기사 (22)



F1cloth · 냥기사 (22)



F1 · 0x72 토트백 (22)

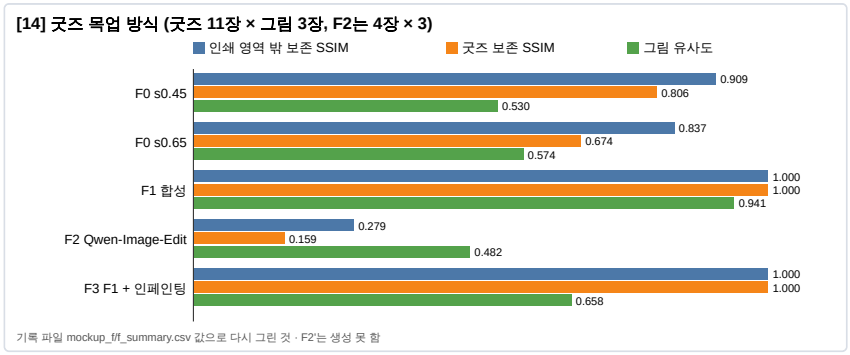


그림 5-4. 목업 방식 비교 (14)

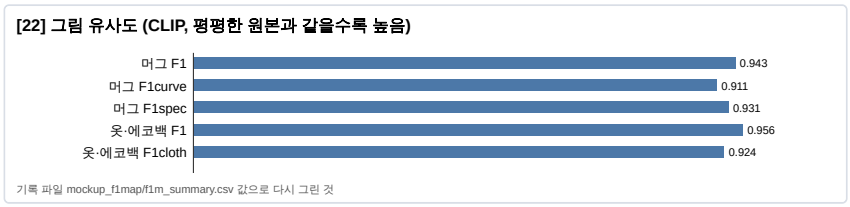


그림 5-5. F1 보강 후보 (22)

- 인쇄 영역: 굿즈 마스크(BiRefNet)의 경계 상자 기준이면 손잡이·옷걸이·옆에 걸린 청바지까지 들어가 어긋나서, 에코백·티셔츠·후드는 몸통 기준으로 다시 잡았습니다(22).
- 실패 기록: F2는 12장 7,718초에 노이즈, FLUX.1 Kontext dev는 디스크 부족과 비상업 라이선스로 제외, F0를 다른 파이프라인과 한 프로세스에 두면 VRAM 24.55GB·첫 장 2,095초 → 단계마다 별도 프로세스 원칙의 근거가 됐습니다(14).

05 · D

기능 D · 굿즈를 지키는 연출 사진

굿즈(실물 사진이나 F에서 고른 목업)는 그대로 두고 배경과 조명만 장면에 맞게 바꿉니다.

방식 (15·16·17)	굿즈 SSIM	색 차이 dC	밝기 변화 dL	문제
D1 마스크 + SDXL 인페인팅	0.999	0.23	0.13	굿즈 테두리 밖에 굿즈를 이어 그림(머그 추가, 큰 가방)
D2 IC-Light 배경 조건	0.842	30.97	10.43	흰 굿즈가 조명색(분홍·주황)으로 바뀜
R1 조명 밝기 비율만 옮김	0.975	0.78	9.67	흰 머그가 회색 (비율 평균 0.56)
R1n 비율을 굿즈 안 평균으로 나눔 (채택)	**0.993**	**0.32**	**3.34**	-
Y 이어 그리기를 줄이는 배경 생성 (채택)	**1.000**	**0**	**0**	**원출력 33장 중 12장 왼쪽 검은 띠**
YR1n Y 배경 + 조명 맞추기 (옵션)	**0.967**	**0.45**	**5.57**	-



입력: F에서 고른 목업 (#25)



출력: YR1n · 카페 (#26)



R1n · desk (16)



Y · desk (16)

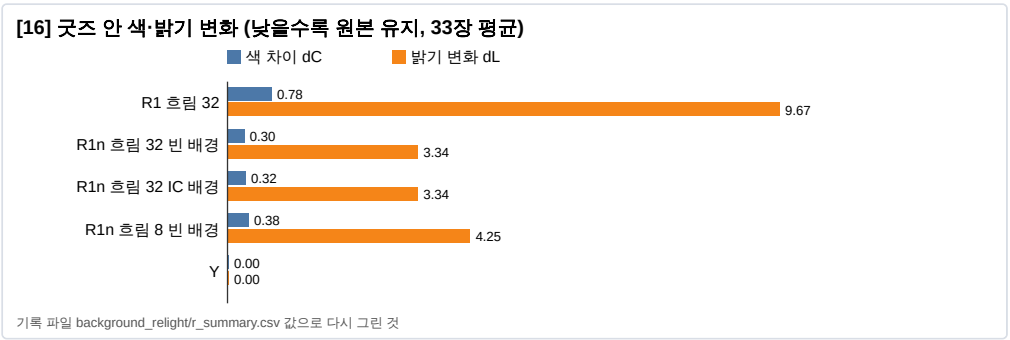


그림 5-6. 굿즈 안 색·밝기 변화 (16)

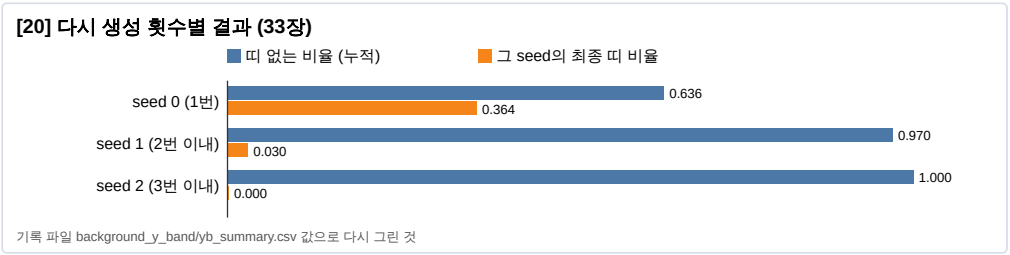


그림 5-7. Y 검은 띠: seed를 바꿔 다시 생성 (20)

- R1n과 Y는 장단점이 달라 둘 다 앱 선택지로 두고, Y에 조명을 맞추는 YR1n을 옵션으로 넣었습니다(10/30). YR1n 설정은 지표가 더 높은 흐림 32 대신 실제 결과를 눈으로 비교해 흐림 8·fill을 골랐습니다(17).
- Y의 검은 띠는 원인을 고치는 대신 seed를 바꿔 최대 3번 다시 생성합니다. 3번 이내 띠 없음 100%, 장당 평균 1.39번 ≈ 3.2초이고, 잘라내기는 폭이 평균 10.7% 줄어 택하지 않았습니다(20).

05 · PAGE

상세 페이지 · 결과를 한 장으로

앞 기능의 결과를 구역별로 골라 가로 860px 게임 홍보 페이지 한 장으로 합칩니다. 구역은 대표 그림+제목 → 게임 소개 → 게임 화면 → 일러스트 → 굿즈 디자인 → 연출 사진 → 굿즈 구매 정보 → 맨 아래(넣지 않음·구매 버튼·출시 예정)이고, 빈 구역 문구는 A와 같은 방식으로 생성합니다.

그리는 방식 (26)

방식	자동 검사 문제	빼고 그린 글자	장당
1번 Pillow	0/30	이모지 3	0.704초
2번 HTML → PNG (채택)	**0/30**	**0**	**1.447초**

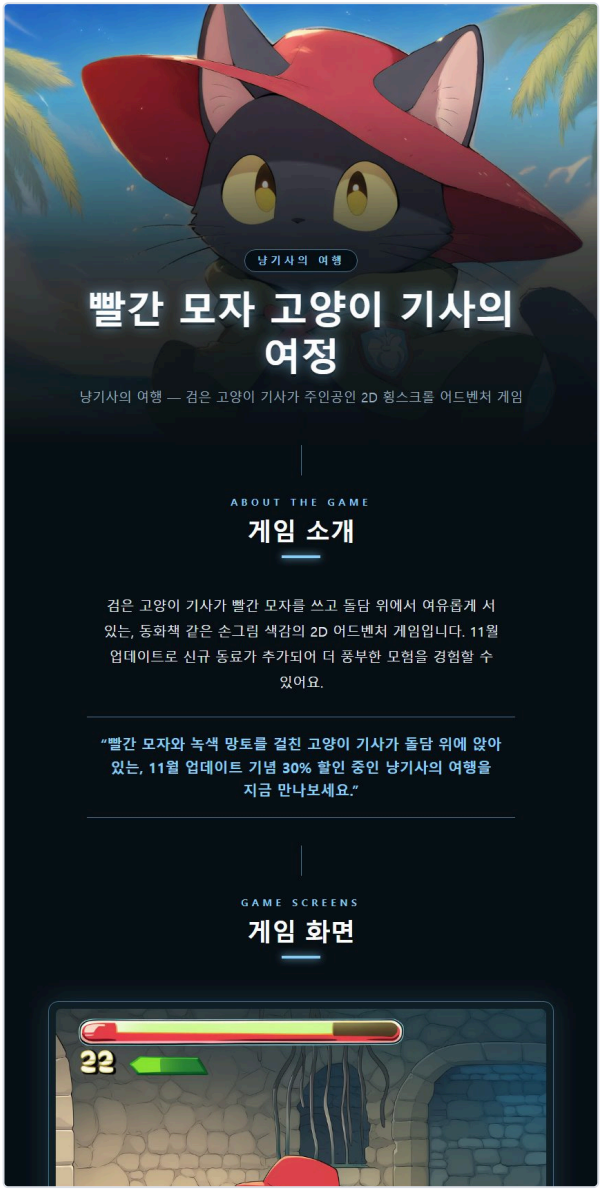
극단 입력 30개(긴 제목, 일러스트 0~9장, 3:1·1:3·64px 그림, 무채색 대표 그림, 빈 값 등)로 페이지 밖·글자 겹침을 자동 검사했습니다. 둘 다 문제 0건이고 이모지를 그리는 HTML을 골랐습니다.

올리는 그림 기준 (27)

긴 변 512px 그림도 큰 칸에서는 1.2~1.8배로 늘어나고, 확대 없이 들어가려면 618~900px가 필요했습니다. 실제 픽셀 확대 비교로 **A 스크린샷·굿즈 사진 512px, 레퍼런스 아트 256px, 가로:세로 3:1~3:4**를 기준으로 정했고, 기준 밖이면 앱이 필요한 크기와 잘라야 할 길이를 안내합니다.

구역 문구 검사 (21)

한 페이지에서 말투가 섞이지 않게 굿즈 디자인·연출 사진 문구에 존댓말 어미 검사를 넣었고, 다시 실행해 규칙 통과 12/12, 해당 문구 6건 모두 존댓말이었습니다(이전 실행은 3건이 반말 "…있어.").



냥기사 상세 페이지 윗부분 (21, page__nai__R1n.png)

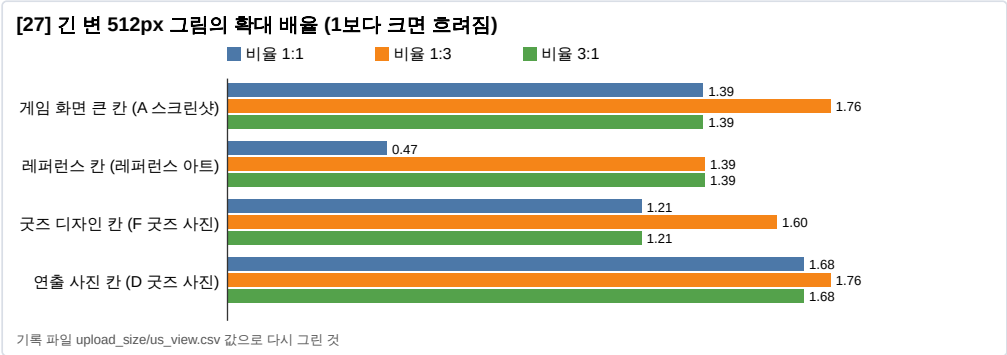


그림 5-8. 긴 변 512px 그림의 칸별 확대 배율 (27)

06

서비스 처리 시간

상세 페이지 한 장에는 문구·그림·배경·마스크·조명 모델이 모두 필요합니다. 요청마다 모델을 새로 올리면 시간 대부분이 모델 로드에 쓰여, 같은 단계의 요청을 모아 모델을 한 번만 올리는 단계별 묶음 처리를 21에서 측정했습니다(게임 3개 × D 방식 2개 = 6페이지).

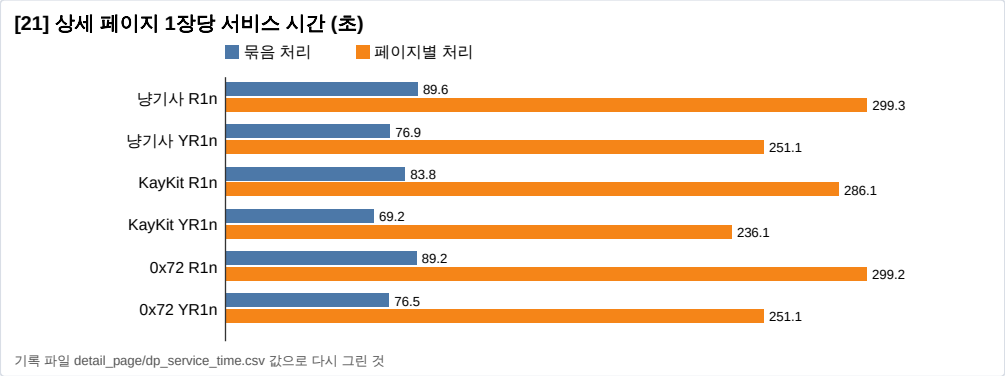


그림 6-1. 페이지 1장당 서비스 시간: 묶음 처리 평균 80.9초, 페이지별 처리 270.5초 (3.35배, 21 dp_service_time.csv)

단계 (21)	로드	항목 평균	최대 VRAM
문구 Qwen3-VL-8B	16.53초	8.83초	18.23GB
C 그림 animagine + IP-Adapter	20.72초	11.23초	13.0GB
빈 배경 SDXL base	17.38초	36.85초	16.87GB
목업 후보 합성 (GPU 없음)	0	0.40초	0
마스크 BiRefNet	4.01초	0.58초	3.12GB
Y 배경	10.67초	3.98초	10.08GB
IC-Light	8.33초	6.66초	4.49GB
합성 Chromium	0.52초	1.43초	0

- 묶음 6장 합 485.2초 중 고정 시간(프로세스 시작·import·모델 로드·종료)이 243.8초(50%)입니다.
- 한 프로세스에 파이프라인 여러 개를 두면 VRAM이 쌓여 극단적으로 느려졌습니다(빈 배경 + F0: 13.1GB → 21.3GB, 5분 넘게 0장). 그래서 단계마다 별도 프로세스를 띄우고, 앱의 워커도 같은 방식으로 만들었습니다.
- 포기한 것: 묶음 6장이 함께 끝나므로 요청 하나가 묶음 전체 시간을 기다립니다. Qwen 4bit·적은 스텝 LoRA 같은 추가 단축은 실험하지 않았습니다.

07

사용성 자체 평가 (가이드 6개 항목)

11/04 앱으로 새 게임 "리치의 모험"을 등록하고 A → B → C → F → D → 상세 페이지를 차례로 만든 시연(12분 53초)을 녹화하고, 앱이 남긴 기록(app.db, stage_times.csv, worker.log)으로 6개 항목을 혼자 평가했습니다(28).

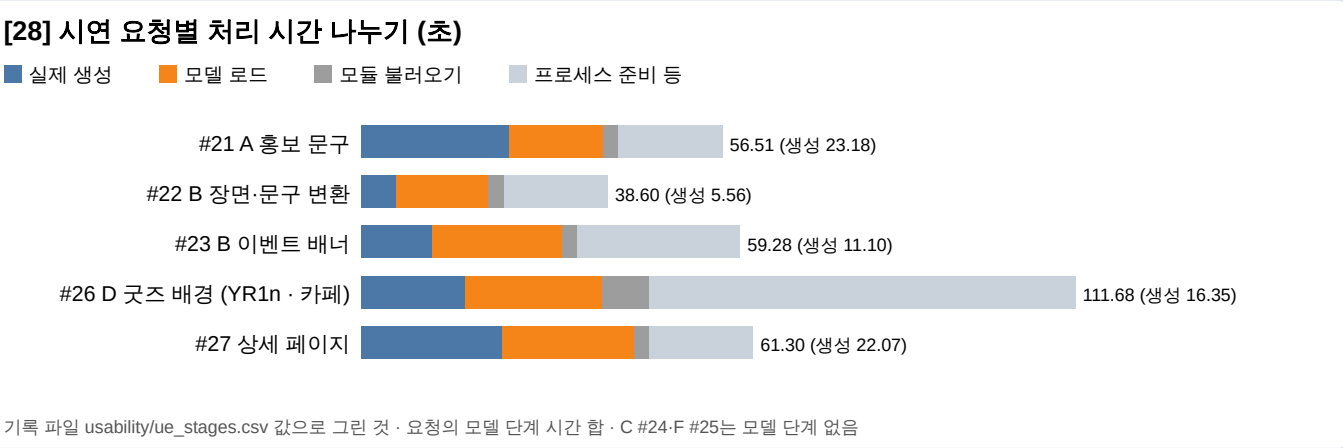


그림 7-1. 시연 요청별 처리 시간 나누기 (요청의 모델 단계 시간 합). 모델 단계 7개 합계 319.79초 중 실제 생성은 70.71초(0.221)이고 나머지는 모델 로드·프로세스 준비입니다(28 ue_stages.csv).

항목	판단	근거
정확도와 적합도	규칙으로 재는 항목(형식·꼭 넣을 문구·판촉 표현·글자 대비)은 코드로 검증. 문구 내용·말투는 이용자가 결과 칸에서 직접 확인·수정	A 결과 문구 칸·상세 페이지 구역 문구 칸 (수정 가능)
일관성과 안정성	내용은 일관됨(#21 후보 5개 모두 같은 주제어·꼭 넣을 문구). 워커 상태 저장은 재시도로 안정화	상태 파일 저장 재시도로 고친 뒤 단계 10번 워커 오류 0건 (worker.log 11/04 18:56~19:02)
응답 속도와 부하 처리	요청당 처리 38.60~111.68초(모델 단계 합), 그중 실제 생성 0.221. 동시 요청 부하는 시험하지 않음	그림 7-1, 21 묶음 처리
유저 중심 인터페이스	기능 사이 흐름은 이어짐(B 배너 그림을 C로, F 목업을 D로, 결과를 상세 페이지 선택지로). 모든 화면 사이드바에 작업 대기열 진행 상황 표시	시연 영상, 앱 사이드바 진행 상황
서비스 완성도	등록부터 상세 페이지까지 한 번에 동작	시연 요청 7건 모두 완료
제어 가능성	입력·옵션·중간 결과(B 장면 태그 확인·수정)로 방향을 바꿀 수 있음	요청 입력값(app.db), 영상 5:45~6:02

평가 뒤 고친 것

- 워커 상태 파일 저장: 실패하면 0.1~0.4초 기다리며 5번까지 다시 해 보고, 그래도 안 되면 그 기록만 건너뜁니다. 수정 코드 워커가 단계 10번을 워커 오류 0건으로 처리했습니다(worker.log 11/04 18:56~19:02).
- 작업 대기열 "끝난 요청 기록 지우기", 요청 번호를 다시 쓰지 않게 한 수정, 앱 창을 닫으면 워커·모델 프로세스도 꺼지게 한 수정, 상세 페이지 맨 아래를 고르는 칸(구매 버튼은 링크가 있을 때만)을 넣었습니다(11/04~11/05).

08

성과 · 한계 · 개선 방향

성과

- **로컬 모델만으로 6개 기능을 하나의 서비스로 연결**했습니다. 게임 등록 → A → B → C → F → D → 상세 페이지가 앱에서 한 번에 동작했습니다(28).
- **후보를 실험으로 비교해 정했습니다.** 실험 노트북 29개, 각 결정마다 비교 대상과 숫자 근거를 남겼고, 결과가 나빠진 결정(F0 → F1)은 바꿨습니다.
- **생성 모델이 못 하는 부분은 코드로 막았습니다.** 문구 규칙 검증과 재요청(A 규칙 전부 통과 KayKit 0.40 → 1.00), 글자 합성(B WCAG 12/12), 원본 픽셀 합성(F 보존 1.000, D 굿즈 SSIM 0.993).
- **서비스 시간을 설계 단계에서 쟀습니다.** 단계별 묶음 처리로 상세 페이지 1장 270.5초 → 80.9초(21), 이 구조를 그대로 앱의 작업 대기열로 옮겼습니다.

한계

한계	내용 (근거)
기다리는 시간	요청당 처리 38.60~111.68초(모델 단계 합) 중 대부분이 모델 로드·준비이고, 묶음 처리는 요청 하나가 묶음 전체를 기다립니다(21·28). 동시 요청 부하는 시험하지 않았습니다.
평가 지표	CLIP 그림체 점수가 KayKit 애니풍을 더 높게 매기는 등 지표가 잡지 못하는 경우가 있어 눈 확인을 함께 했습니다(07). 사용성 평가는 다른 사용자 없이 혼자 했습니다(28).
D 범위	D는 기본 방식 + 조명 옮기기까지만 구현했습니다(10/28 범위 결정).

개선 방향

- 모델 로드 시간 단축: 21에서 실험하지 않은 후보(Qwen 4bit, 적은 스텝 LoRA)를 비교하는 것.

09

진행 과정

- **10/15~10/16** 프로젝트 시작, 사용자·기능 정의, 예시 게임 데이터 확보(NAI 800장, KayKit·0x72 각 520장)
- **10/20** 예시 게임 3개 확정, 텍스트만으로 이미지 모델 13개 비교(01) → IP-Adapter 비교 필요
- **10/21** IP-Adapter 베이스 모델 6개 비교 → animagine_xl4 (05)
- **10/22** 로컬 VLM 4개 비교 → Qwen3-VL-8B로 A·B (08), 배너 글자는 Pillow 합성, 냥기사 LoRA는 Zero 학습 + 트리거 캡션 (06)
- **10/23** 문구 재생성은 샘플링 후보 여러 개 방식 (08)
- **10/26** 배너 문제 4가지 수정, rating 태그, 판촉 표현 재요청 (07)
- **10/27** 후보 4개 (09), KayKit 꼭 넣을 문구 개선 (10), LoRA 장수 비교 (11), IP-Adapter 켬 (12)
- **10/28** F 목업 방식 비교 → F0 (14), D 방식 비교 (15), F를 깊게·D는 기본만으로 범위 결정
- **10/30** ③ 세 게임 공통 (13), D R1n·Y·YR1n (16·17·20), 냥기사 LoRA 50장 + IP-Adapter (18), 상세 페이지 설계 (21)
- **11/02** F0 → F1 합성 계열로 교체, 후보를 모두 만들어 고르게 함 (22), 상세 페이지 묶음 처리 측정 (21)
- **11/03** 후보 8개 비교 (23), 새 게임 기본 설정 확인 (24), 재요청을 편집 지시로 (25)
- **11/04** 상세 페이지 HTML → PNG (26), 올리는 그림 기준 (27), 시연 녹화·사용성 자체 평가 (28), 워커 저장 오류 수정, start.bat 한 번 클릭 실행
- **11/05** 요청 번호 재사용 수정, 앱을 닫으면 워커 종료, 상세 페이지 맨 아래 고르기, Notion 실험 정리

출처: Notion 업무일지·데일리 브리핑, "실험 정리 (보고서용)" 결정 날짜

APPENDIX A

실험 노트북 00~28 목록

번호	기능	확인한 것	핵심 결과	이어진 결정
00	공통	실행 환경 설정, 패키지 버전 기록	torch 2.14.1+cu130, diffusers 0.41.0.dev0, transformers 5.20.1	실험 공통 패키지 기준
01	B·C·D·F	이미지 생성 모델 13개, 텍스트만	SDXL 계열 3개가 장당 약 15.2초·약 9GB로 가장 가벼움, IP-Adapter 로드 6개, 모두 게임 그림체에서 벗어남	6개를 05 후보로
02	A	Claude Haiku·Sonnet·Opus 기준선	45개 중 "모험" 44, "떠나" 44, "~보세요" 43	Claude는 기준선으로만
03	데이터	1,840장 EDA, 무지 굿즈 사진 수집	NAI 알파는 생성 정보, 밝기·채도 기준은 장면 밝기를 거름, 굿즈 11장	참조 이미지 3장
04	C	냥기사 LoRA 데이터 NovelAI 생성	800/800장	06·11 학습 데이터
05	C (B)	IP-Adapter 베이스 6개, 162장	animagine_xl4 냥기사 0.715~0.728, 장당 약 17초	세 게임 animagine_xl4
06	C	LoRA 체크포인트 2 × 캡션 2	Zero 학습 → 4.0 적용이 8개 조건 모두 높음	Zero + 트리거 캡션
07	B	글자 합성, 요청 변환, 문제 수정, rating, 판촉	WCAG 12/12, 노출 5/8 → 0/8, 판촉 첫 실패 3/36 → 0/36	Pillow 합성, 게임별 프롬프트, safe, 재요청
08	A·B	로컬 VLM 4개, 꼭 넣을 문구, 재생성	Qwen 형식 1.00·상투 0.07, B 1.00·4.43초	Qwen 하나로 A·B
09	A	샘플링 후보 수	후보 4개 0.58 (2개 0.42), KayKit만 0.40	후보 4개
10	A	KayKit 꼭 넣을 문구	③ 1.00 (기준 0.40)	③ 채택
11	C	LoRA 데이터 50~800장	그림체 0.741~0.760, 차이가 표준편차 안	50장으로 충분
12	C	IP-Adapter 켬/끔	KayKit 0.552 → 0.646 등	IP-Adapter 켬
13	A	③을 냥기사·0x72에	냥기사 0.867, 0x72 0.800	③ 세 게임 공통
14	F	F0·F1·F2·F3	F1 보존 1.000·0.3초, F2 596초	F0 → (11/02) F1
15	D	D1 인페인팅, D2 IC-Light	D1 이어 그림, D2 색 바꿈	16에서 조명만 옮기기
16	D	R1·R1n, Y	R1n SSIM 0.993, Y 픽셀 그대로·2.31초	R1n·Y 선택지
17	D	YR1n	흐림 8·fill SSIM 0.967	YR1n 옵션
18	C	LoRA + IP-Adapter	0.737 (IP-Adapter만 0.718), 50장 = 800장	zero_trigger_n50
19	A	재요청 조사 수정	냥기사 0.800, 수정 전과 같음	변화 없음
20	D	Y 검은 띠	3번 이내 100%, 평균 1.39번	최대 3번 다시 생성
21	상세 페이지	묶음 처리·서비스 시간	80.9초 vs 270.5초, 구역 문구 12/12	앱 작업 대기열(워커)
22	F	F1 보강, 인쇄 영역	장당 0.31~0.35초, 14 재현 차이 0	후보 모두 만들고 고름
23	A	후보 4 → 8개	평균 0.889 → 0.844	4개 유지
24	C	새 게임 기본 설정	그림체 −0.031~−0.079, 주인공 바꿈	게임 등록에 그림체 태그·LoRA 칸
25	A	따로 붙는 실패 후보 6개	편집 지시 0.956 (0x72 1.000)	편집 지시 적용
26	상세 페이지	Pillow vs HTML, 극단 입력 30개	둘 다 문제 0건, HTML은 이모지도 그림	HTML → PNG
27	앱	올리는 그림 크기·비율	512px도 1.2~1.8배 확대	512·256px, 3:1~3:4
28	앱	사용성 자체 평가	요청 7건 완료, 실제 생성 0.221	워커 상태 저장 재시도

자세한 비교표·실패 조건은 Notion "실험 정리 (보고서용)"와 개별 실험 페이지 [01]~[28]

APPENDIX B

실행 환경과 실행 방법

환경

항목	내용
하드웨어	NVIDIA GeForce RTX 3090 24GB, Windows
주요 패키지 (requirements.txt)	torch 2.14.1+cu130, diffusers git `efe46a5b` (0.41.0.dev0), transformers 5.20.1, accelerate 1.15.0, peft 0.21.2, streamlit 1.65.0, pillow 12.3.0, playwright(Chromium)
모델	Qwen/Qwen3-VL-8B-Instruct, cagliostrolab/animagine-xl-4.0 (+ 4.0-zero로 학습한 LoRA), h94/IP-Adapter, SDXL base, ZhengPeng7/BiRefNet, yahoo-inc/photo-background-generation, IC-Light iclight_sd15_fbc + realistic-vision-v51

실행

- **앱 한 번 클릭 실행**: 프로젝트 폴더의 `start.bat`. 처음 실행하면 프로젝트 폴더에 `.venv`를 만들어 앱 전용 패키지(`requirements-app.txt`)와 Chromium을 설치한 뒤 앱을 켭니다. 이 PC에서 첫 설치는 333초가 걸렸습니다. 모델은 처음 쓸 때 Hugging Face에서 받습니다.
- **실험 노트북 환경**: `pip install -r requirements.txt` 뒤 `python -m playwright install chromium`.
- **앱 직접 실행**: `streamlit run src/app/streamlit_app.py` (워커는 앱이 자동으로 띄움). 앱과 실험 노트북은 같은 GPU를 쓰므로 동시에 돌리지 않습니다.

폴더 구조

`experiments/`	실험 노트북 00~28
`src/app/`	서비스 앱 (Streamlit 화면 + 대기열 워커)
`src/train/`	LoRA 학습 스크립트
`outputs/`	실험 기록 파일·앱 결과
`data/raw/`, `models/lora/`	원본 데이터, 학습한 LoRA

APPENDIX C

출처 간 숫자 차이

같은 항목이 기록마다 다르게 적힌 경우와, 본문에서 어느 값을 썼는지입니다.

항목	출처 1	출처 2	본문에서 쓴 값
D2 굿즈 SSIM	0.806 (15 d_summary.csv)	0.842 (16 r_summary.csv)	0.842: 15는 결과를 원본 틀로 늘려 위치가 어긋났고 16은 위치를 맞춤
F2 장당 시간	596초 (f_summary.csv)	12장 7,718초 − 로드 535초 ≈ 598.6초/장	596초
Qwen 최대 VRAM	16.8GB (08)	17.27GB (08 재생성) ~ 18.23GB (21)	실험마다 생성 길이·후보 수가 달라 범위로 표기
Y 로드 시간	34.2초 (20)	102.1초 (16, 다운로드 포함), 10.67초 (21)	21 값(10.67초)